1. Práctica 8: Buses de Expansión - I2C y pantalla táctil

1.1. Objetivos

- Entender el funcionamiento básico del bus I2C.
- Programar un driver para la comunicación con la pantalla táctil a través de I2C.

1.2. Fundamentos teóricos

Si bien los procesadores cuentan con periféricos internos con funcionalidades muy amplias (temporización, controles de memoria, interrupciones, protocolos básicos de comunicación...), hay ciertas funciones que no pueden tener integradas. Por ejemplo, sería muy complicado tener un periférico de pantalla integrado, ya que cada pantalla es un mundo y habría que limitar los dispositivos soportados. Lo mismo ocurriría con audio, teclados, ratones, actuadores...

Es decir, para actuar con el mundo .exterior.al procesador, no podemos contar con periféricos específicos. En su lugar, se suelen ofrecer periféricos genéricos que, utilizando protocolos estandarizados, se pueden conectar con una gran cantidad de elementos externos. Así, el trabajo de controlar esos periféricos externos será ahora del programador *software*, mediante la escritura de los *drivers* necesarios.

En nuestro caso, vamos a utilizar el bus **I2C** para la conexión con el panel táctil. Este bus utiliza dos líneas, **SCL** para transmitir una señal de reloj, y **SDA** para transmitir los datos. Permite, a través de identificación mediante direcciones (enviando parejas dirección-dato por la línea **SDA**), la interconexión de múltiples dispositivos. Su extrema sencillez y flexibilidad han hecho que se haya popularizado ampliamente.

1.2.1. Conexiones en nuestra placa

Podemos ver el pinout completo de las conexiones procesador-pantalla táctil en `um2140 6.8`. El conector a la pantalla táctil tiene señales **SCL**, **SDA**, **INT**, **RESET**, además de varias conexiones para alimentación y tierra. Estas últimas son pasivas y no requieren de control por parte del procesador.

Las que sí requieren de control, se especifican en `um2140 Appendix A`. Ahí, podemos ver las conexiones que se especifican en la Tabla 1, con un esquema ilustrativo en la Figura 1.

Pin	Function	Connection
PA8	I2C3 SCL	CTP SCL
PH8	I2C3 SDA	CTP SDA
PH9	GPIO OUTPUT	CTP RST
PI9	GPIO EXTI9	CTP INT

Tabla 1: Conexiones para la pantalla. CTP: *Connector Touch Panel*. Los pines PA8 y PH8 no vienen completamente especificados en `um2140`, pero se incluyen aquí por completitud.

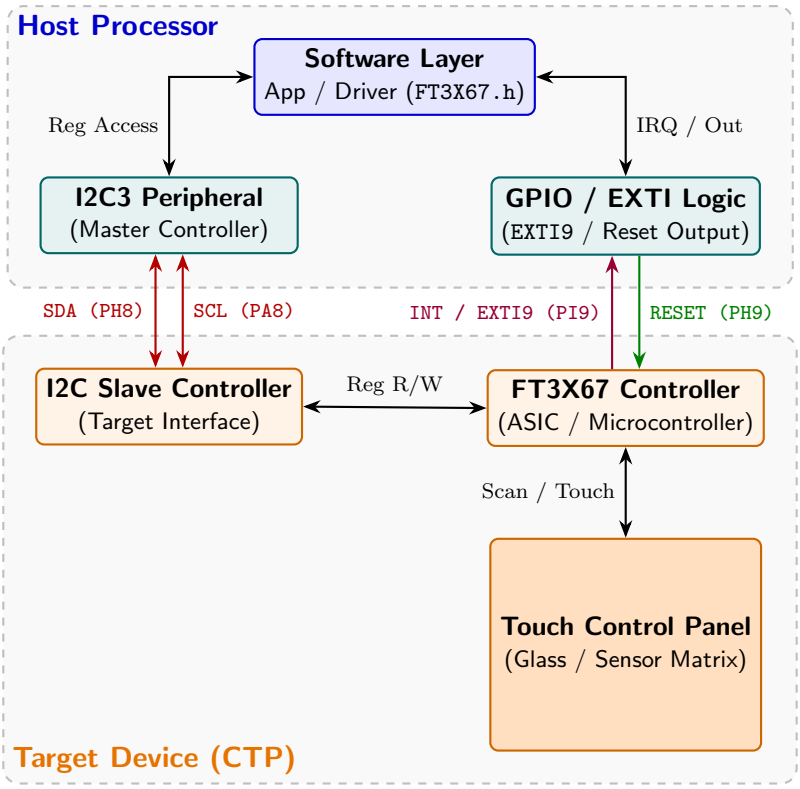


Figura 1: Esquema de comunicación entre el procesador principal, el controlador I2C y el ASIC del panel táctil.

1.2.2. Funcionamiento del periférico I2C

El periférico que controla el bus **I2C** se encuentra descrito en `rm0431 26`. Más allá de la configuración concreta de los registros internos, para controlar los parámetros de **I2C** como frecuencia, funcionamiento y demás, el periférico ofrece dos funciones principales:

1. Enviar un dato a un registro concreto de un dispositivo externo concreto.
2. Recibir un dato de un registro concreto de un dispositivo externo concreto.

Esto se hará más adelante mediante sendas funciones de escritura y lectura (driver) que utilizará el `I2C` para la comunicación con el dispositivo externo de panel táctil. Estas funciones, interactuando con los registros internos del controlador `I2C`, harán transparente para el usuario los detalles de transacción dentro del bus.

1.3. Desarrollo de la práctica

Llegados a este punto, sólo nos queda comunicarnos con el panel táctil. Haremos una aplicación donde el usuario "pintará" con el dedo en la pantalla. Para ello, partiremos de la práctica anterior, donde ya teníamos el controlador de la pantalla funcionando. En lugar de pintarse aleatoriamente, ahora controlaremos nosotros el pintado. Para ello, habrá que hacer lo siguiente:

- Configurar el bus I2C para poder comunicarnos con el panel táctil (CTP).

- Activar las interrupciones adecuadas para recibir eventos del CTP.
- Crear el driver que nos permita leer los eventos y transformarlos a coordenadas para pintar en la pantalla.

Vamos por orden:

1.3.1. Configuración del bus I2C

Lo primero que debemos hacer es activar los pines adecuados para la comunicación. En primer lugar, los pines **SCL** y **SDA** (PA8 y PH8) deben configurarse en modo alternativo con función 4 (esto los conecta al I2C3, ver `ds11853 Table 12`), modo open drain, sin pull up y velocidad alta. Además, hay que asegurarse de activar `GPIOAEN` y `GPIOHEN` en `AHB1ENR` (`rm0431 5.3.10`).

Por otro lado, debemos asegurarnos que el CTP no está en modo RESET, para ello, debemos activar el GPIO para el pin PH9 (Correspondiente a CTP RESET (Tabla 1)). Este pin debe configurarse en modo salida, push-pull sin pullup, y velocidad baja. A continuación debemos escribir un "1" en él para desactivar el RESET (que es en lógica inversa).

```
void I2C3_Pins_Init() {
    //Pin A8 tiene I2C_SCL
    // ... completar
    //Pin H8 tiene I2C_SDA
    // ... completar
    //Pin H9 para el reset del CTP
    // ... completar
}
```

Con esto los pines ya están activos, y nos toca configurar el periférico I2C. Las definiciones de los registros que contiene se pueden ver en `rm0431 26.9`, y la dirección base se puede encontrar en `rm0431 1.6.2`. Se ofrece a continuación el código para `bsp.h` con las definiciones y algunas funciones predefinidas creadas:

```
#define I2C3_BASE_ADDR 0x40005C00UL

typedef struct
{
    volatile uint32_t CR1;      /* Control register 1,            */
    volatile uint32_t CR2;      /* Control register 2,            */
    volatile uint32_t OAR1;     /* Own address 1 register,        */
    volatile uint32_t OAR2;     /* Own address 2 register,        */
    volatile uint32_t TIMINGR;  /* Timing register,               */
    volatile uint32_t TIMEOUTR; /* Timeout register,              */
    volatile uint32_t ISR;      /* Interrupt and status register, */
    volatile uint32_t ICR;      /* Interrupt clear register,      */
    volatile uint32_t PECR;     /* PEC register,                  */
    volatile uint32_t RXDR;     /* Receive data register,         */
    volatile uint32_t TXDR;     /* Transmit data register,        */
} I2C_TypeDef;

#define I2C3                ((I2C_TypeDef *) I2C3_BASE_ADDR)

static inline void I2C_Disable(I2C_TypeDef * I2Cx) {
    I2Cx->CR1 &= ~(0x1);
}

```

```
24 static inline void I2C_Enable(I2C_TypeDef * I2Cx) {
25     I2Cx->CR1 |=  (0x1);
26 }
27
28 static inline void I2C_Set_Timing(I2C_TypeDef * I2Cx, uint32_t presc, uint32_t
       scldel, uint32_t sdadel, uint32_t sclh, uint32_t scll) {
29     I2Cx->TIMINGR = (presc << 28) | (scldel << 20) | (sdadel << 16) | (sclh <<
           8) | (scll);
30 }
```

En lo que a nosotros respecta, debemos utilizar estas funciones para inicializar el periférico `I2C3` (`rm0431` 26). En `rm0431 26.4.5`, tenemos un diagrama que nos indica cómo proceder para configurar el controlador I2C. Como no necesitamos todas las funciones, lo haremos algo más sencillo.

Lo primero será activar el periférico `I2C3` desde `APB1ENR` (`rm0431 5.3.13`), para a continuación aplicar una secuencia de reseteo desde `APB1RST` (`rm0431 5.3.8`) consistente en escribir un 1 y luego un 0 sobre el bit de `I2C3`. Tras esto, ya podemos configurar el periférico. Para ello se inhabilita (ya desde dentro de su registro `I2C_CR1`), se configura el temporizador `I2C_TIMINGR`, y se vuelve a habilitar.

Para la temporización, hay numerosos campos. Tenemos que configurarlos teniendo en cuenta que la frecuencia base son 16MHz. La frecuencia objetivo que queremos son 400KHz. Hay muchas maneras de conseguirla, tenemos que tener en cuenta que la fórmula es la siguiente:

$$f_{obj} = \frac{f_{base}}{(PRESC + 1) * (SCLL + 1 + SCLH + 1)}$$

`SCLL` y `SCLH` indican el tiempo que la señal de reloj pasará, respectivamente, a baja (low) y alta (high). Podemos ponerlos iguales para conseguir una señal cuadrada. El valor de `PRESC` es un preescaler que nos divide la señal de reloj por si fuera muy alta. En nuestro caso, podemos dejar el prescaler a 0, y poner los otros dos registros a 19, para conseguir una onda cuadrada exacta a 400KHz. Los valores de `SCLDEL` y `SDADEL` controlan los desfases entre las subidas y bajadas de **SCL** y **SDA**. En general son valores bastante flexibles, aunque sus detalles son complejos y pueden explorarse más en `rm0431 26.9.5` y `rm0431 26.4.5`. Para nuestro caso, con 3 y 1 respectivamente debería funcionar.

```
1  void I2C3_Reg_Config() {
2      //Enable I2C3 and reset cycle
3      // ... completar
4
5      //Configure I2C
6      I2C_Disable(I2C3);
7      // Set Timing
8      // ... completar
9      I2C_Enable(I2C3);
10 }
```

1.3.2. Interrupciones del CTP

A continuación debemos activar las interrupciones del panel táctil. Como vimos en Tabla 1 tiene un pin dedicado para interrumpir al procesador, a través de la línea de interrupción externa `EXTI9`. Debemos configurar el procesador para atender esa petición. Recordemos que cada

línea de interrupción externa `EXTIx` puede conectarse a cualquiera de los puertos (PA, PB, ...) (rm0431 10). En este caso, como interrumpimos desde `PI9`, debemos configurar `EXTI9` para que atienda al puerto `I` en modo falling edge. Además, debemos indicar desde `SYSCFG` en el registro `EXTICR` (rm0431 7.2.3) que el puerto I (8) es el que se atiende en `EXTI9`, y activar `SYSCFG` desde el bit de enable de `APB2ENR` (rm0431 5.3.19). Finalmente, tenemos que activar la interrupción correspondiente en el `NVIC` (rm0431 9.1.2).

Hay que tener en cuenta que las interrupciones externas están agrupadas (en concreto, de la 5 a la 9 se atienden juntas (rm0431 9.1.2) a través de la línea de interrupción número 23 del procesador). Así que, en nuestra rutina de interrupción, tendremos que asegurarnos de que fue la interrupción externa 9 la que causó la interrupción a través del registro `PR` de `EXTI` (rm0431 10.9.6)

```
void TS_INT_Init(void) {
    // ... completar activar GPIO PI9 como entrada pull-up, velocidad rápida

    /* Enable and Map EXTI Line 9 to GPIOI (0b1000 = Port I) */
    // ... completar activar SYSCFGEN en APB2ENR
    // ... configurar int 9 para atenderse desde PI en EXTICR
    // ... activar interrupción EXTI9 en EXTI
    // ... habilitar modo falling edge y deshabilitar rising edge
    // ... activar interupción 23 (EXTI9_5) en NVIC
}

void EXTI9_5_IRQHandler(void) {
    // ... completar comprobar bit 9 de EXTI PR
    if () {
        // ... limpiar bit 9 de EXTI PR
        g_ts_interrupt_flag = 1;    // Notify main thread
    }
}
```

Con esto, ya tendremos las interrupciones disparando ante eventos del panel táctil. De hecho, puedes probar a ejecutar el código con un *breakpoint* en la interrupción, y deberías ver que se dispara al tocar la pantalla, aunque no recojamos dato alguno.

1.3.3. Driver del I2C

Nos falta leer los datos correspondientes a la posición, y hacer algo con ellos. Para ello necesitamos dos cosas:

- Un *driver* para el periférico `I2C` que nos permita leer/escribir a través del bus (programaremos ambas funciones, aunque sólo utilizaremos la primera).
- Otro *driver* que, utilizando el primero, se comunique con el panel táctil para leer la posición de la pulsación.

Lo primero de todo es crear el driver para `I2C` (rm0431 26). Vamos a empezar por unas cuantas funciones auxiliares para el control de los diversos registros internos. Analízalas bien, viendo los registros que utilizan y el manual, para comprender su funcionamiento:

```
static inline void I2C_Write(I2C_TypeDef *I2Cx, uint8_t data) {
    // Wait for TXDR to be empty (TXIS == 0)
    while (!(I2Cx->ISR & (1U << 1U)));
```

```
    // Load byte into Transmit Data Register
    I2Cx->TXDR = data;
}

static inline uint8_t I2C_Read(I2C_TypeDef *I2Cx) {
    // Wait for RXNE (Receive Buffer Not Empty flag)
    while (!(I2Cx->ISR & (1U << 2U)));
    // Read byte from Receive Data Register
    return (uint8_t)(I2Cx->RXDR & 0xFFU);
}

static inline void I2C_Wait_TC(I2C_TypeDef *I2Cx) {
    // Wait for TC (Transfer Complete flag)
    while (!(I2Cx->ISR & (1U << 6U)));
}

static inline void I2C_Stop(I2C_TypeDef *I2Cx) {
    // Generate STOP condition (STOP bit 14 in CR2)
    I2Cx->CR2 |= (1U << 14U);
    // Wait for STOPF (Stop detection flag bit 5 in ISR)
    while (!(I2Cx->ISR & (1U << 5U)));
    // Clear STOPF flag by writing to ICR (STOPCF bit 5 in ICR)
    I2Cx->ICR |= (1U << 5U);
}
```

Uno de los registros más importantes es `CR2` (`rm0431 26.9.2`). Nos va a permitir configurar la transferencia que queremos realizar (dirección destino, número de bytes a trasferir, si es escritura o lectura, y algunas funciones más que no utilizaremos). Estúdialo bien, e implementa la función `I2C_Start`:

```
typedef enum {
    I2C_WRITE = 0U,
    I2C_READ  = 1U
} I2C_Direction;

static inline void I2C_Start(I2C_TypeDef *I2Cx, uint8_t slave_addr,
    I2C_Direction direction, uint32_t len) {
    uint32_t cr2 = 0;

    // ... completar Set 7-bit Slave Address (SADD[7:1])
    // ... completar Set Transfer Length (NBYTES[23:16])
    // ... completar Set Direction (RD_WRN bit 10: 0 = Write, 1 = Read)
    // ... completar Generate START Condition (START bit 13)

    // Apply configuration to hardware
    I2Cx->CR2 = cr2;
}
```

Una vez tenemos estas funciones auxiliares, ya podemos programar la escritura y lectura a periféricos a través del bus I2C. Recordemos de **??** el funcionamiento general de I2C. Más allá de no necesitar controlar los ACK (ya que se generarán solos), debemos programar el resto de la lógica siguiendo los pasos allí indicados. Se proporciona el esqueleto a continuación:

```
/* Writes 1 byte to a specific register inside the sensor */
void I2C_Reg_Write(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg, uint8_t
    val) {
    //... completar Issue START + set up Address (Write Mode, 2 bytes)
    //... completar Write Target Register Address
    //... completar Write Value
    //... completar Wait for transfer complete (TC)
    //... completar issue STOP condition
```

```
}

/* Reads 'len' sequential bytes starting from 'reg' */
void I2C_Reg_Read_Seq(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg,
    uint8_t *dest, uint32_t len) {
    // ================= PHASE 1: WRITE REGISTER ADDRESS =================
    // ... completar Issue START + set up Address (Write Mode, 1 Byte)
    // ... completar Write target register address
    // ... completar Wait for Transfer complete (TC)

    // ================= PHASE 2: READ DATA PAYLOAD =================
    // ... completar Issue START + set up Address (Read Mode, 'len' Bytes)
    // ... completar Read 'len' bytes from the sensor
    // ... completar Wait for Transfer complete (TC)
    // ... completar issue STOP condition
}
```

1.3.4. Driver del CTP

Finalmente, y con las funciones `I2C` creadas, estamos listos para leer los registros del panel táctil con la información correspondiente a las coordenadas de la pulsación. La documentación del mapa de registros interno del panel, y sus valores es bastante escasa. Pero sí disponemos del driver oficial de ST en `https://github.com/STMicroelectronics/stm32-ft3x67/tree/main`. Consultando el fichero `ft3x67.h` podemos ver el mapa de registros, así como los diversos códigos de control y respuesta. Para simplificar el proceso y no hacer un driver tan complejo, únicamente vamos a fijarnos en los siguientes registros internos mostrados en la Tabla 2 y Tabla 3.

Addr	Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0x02	FT3X67_TD_STAT_REG	Reserved [7:4]				**Touch Points [3:0]** (0–2)			
0x03	FT3X67_P1_XH_REG	**Event [7:6]**		Res [5:4]		**P1_X[11:8]** (MSB)			
0x04	FT3X67_P1_XL_REG	**P1_X[7:0]** (LSB)							
0x05	FT3X67_P1_YH_REG	Reserved [7:4]				**P1_Y[11:8]** (MSB)			
0x06	FT3X67_P1_YL_REG	**P1_Y[7:0]** (LSB)							
0x07	FT3X67_P1_WEIGHT_REG	**P1_WEIGHT[7:0]**							
0x08	FT3X67_P1_MISC_REG	**P1_MISC[7:0]**							
0x09	FT3X67_P2_XH_REG	**Event [7:6]**		Res [5:4]		**P2_X[11:8]** (MSB)			
0x0A	FT3X67_P2_XL_REG	**P2_X[7:0]** (LSB)							
0x0B	FT3X67_P2_YH_REG	Reserved [7:4]				**P2_Y[11:8]** (MSB)			
0x0C	FT3X67_P2_YL_REG	**P2_Y[7:0]** (LSB)							
0x0D	FT3X67_P2_WEIGHT_REG	**P2_WEIGHT[7:0]**							
0x0E	FT3X67_P2_MISC_REG	**P2_MISC[7:0]**							

Tabla 2: Disposición de los registros en el controlador de panel táctil FT3X67

Como de momento sólo nos interesa leer una coordenada, podemos leer los primeros 5 registros y será suficiente. Deberemos crear un nuevo fichero llamado `FT3x67.h` que contenga la lógica para hacerlo:

Field / Mask	Bits	Name	Values / Description
TD_STAT_MASK	[3:0]	Active Touch Points	Number of active touch points detected (0 to 2).
TOUCH_EVT_FLAG	[7:6]	Event Flag	0x00 (00b): Press Down 0x01 (01b): Lift Up 0x02 (10b): Contact 0x03 (11b): No Event
POS_MSB_MASK	[3:0]	Position MSB	Upper 4 bits of X or Y coordinate (bits [11:8]).
XL / YL	[7:0]	Position LSB	Lower 8 bits of X or Y coordinate (bits [7:0]).
WEIGHT	[7:0]	Touch Weight	Pressure / weight value of touch contact.
MISC	[7:0]	Miscellaneous	Touch area / extra touch panel metrics.

Tabla 3: Definiciones y valores para los registros del FT3X67

```c
/*
* FT3X67.h
*
*  Created on: Aug 11, 2026
*      Author: dani
*/

#ifndef FT3X67_H_
#define FT3X67_H_

#define TS_I2C_7BIT_ADDR 0x38
#define FT_REG_TD_STATUS 0x02

typedef struct {
    uint16_t x;
    uint16_t y;
    uint8_t  touch_detected;
} TS_State_t;

/**
* @brief  Reads the current touch coordinates from the FT3X67.
* @param  ts: Pointer to TS_State_t structure to store parsed coordinates
* @return 1 if touch is valid, 0 otherwise
*/
uint8_t FT3X67_GetTouchCoordinates(TS_State_t *ts) {
    uint8_t buffer[5];

    /* Read 5 consecutive registers starting at TD_STATUS (0x02 to 0x06) */
    // ... completar leer utilizando I2C_Reg_Read_Seq 5 bytes

    uint8_t num_touches = buffer[0] & 0x0F;

    if (num_touches > 0 && num_touches <= 2) {
        // ... completar parsear coordenadas en ts->x y ts->y

        ts->touch_detected = 1;
        return 1;
    }

    ts->touch_detected = 0;
    return 0;
}
```

```
43
44 #endif /* FT3X67_H_ */
```

1.3.5. Integración

Finalmente, ya estamos listos para integrar todo en nuestro código `main.c`. Tras inicializar el I2C y el panel táctil con las funciones `I2C3_Init()` y `TS_INT_Init()` podremos entrar en un bucle infinito que espere el aviso de interrupción del panel táctil antes de leerlo:

```
1  // ... código de inicialización de la pantalla (práctica anterior)
2
3  I2C3_Init();
4  TS_INT_Init();
5
6  while (1) {
7      /* Check if the touchscreen interrupt fired */
8      if (g_ts_interrupt_flag) {
9          g_ts_interrupt_flag = 0; // Reset software flag
10
11         // ... completar obtener coordenada utilizando
12         // ... el driver FT3X67 y pintarla en la pantalla
13         // ... hay que rotarla pues las coordenadas
14         // ... del panel y la pantalla no coinciden
15     }
16 }
```

1.4. Para hacer más

Investiga el repositorio oficial `https://github.com/STMicroelectronics/stm32-ft3x67/tree/main` para el driver del panel táctil, e intenta implementar detección de gestos. También puedes extender la lógica de tu programa para que, en lugar de pintar sólo los puntos que tocas, vaya haciendo una línea entre ellos (verás que, de lo contrario, los puntos no son perfectamente contiguos).